

Congress Trading Signal

La couche données (Semaines 1–2)

Extraction, identification, enrichissement et validation honnête des déclarations boursières du Congrès américain — Chambre *et* Sénat, 2020–2026

90 487 transactions (House **81 646** + Sénat **8 841**) extraites de bout en bout des sources officielles (*House Clerk*, Sénat *eFD*), piste électronique **et** scannée (OCR Claude Vision) ; identité résolue au `bioguide_id`, table **12/12 champs**, validée contre Quiver (vérification externe, jamais réinjectée), et **reproductible** (golden octet-à-octet). Ce document se lit *main dans la main* avec le dépôt : chaque mécanisme cite le `fichier:fonction` réel.

Alice Lemaire

26 juin 2026

Périmètre : couche données (Semaines 1–2). La stratégie/backtest (Semaines 3–4) est un travail de recherche séparé, en cours, et hors-périmètre de ce rapport.

Résumé

Les membres du Congrès américain déclarent leurs transactions boursières sous le *STOCK Act* ; ces déclarations, exploitées par une stratégie de *copy-trading*, exigent au préalable une **couche données** propre, traçable et honnêtement validée. Ce rapport présente cette couche, livrée sur les Semaines 1–2. Le pipeline reconstruit **90 487 transactions** sur deux chambres et sept ans (2020–2026) : **81 646** pour la Chambre (32 676 électronique + 48 970 OCR) et **8 841** pour le Sénat (7 161 électronique + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (`bioguide_id`) : **99,99 %** des lignes Chambre, **100 %** au Sénat. Chaque ligne est enrichie (ticker, secteur **GICS**→**ETF SPDR**, type d'actif, montant *midpoint*, ancienneté), aboutissant à la **table 12/12 champs** demandée par Ramify. La validation externe contre Quiver — **jamais réinjectée** — donne une concordance transaction-niveau de **85,9 %/an** (Chambre, un plancher) et **98–100 %/an** (Sénat), avec **9** et **0** vrais-absents après explication ; le papier scanné, invisible aux agrégateurs, est une **valeur ajoutée** propre, validée en interne. La correction est prouvée par un **golden octet-à-octet** (108 fichiers Chambre, 69 Sénat) plus des reproductions fonction-par-fonction. Le rapport rend compte des **hypothèses méthodologiques révisées** en cours de route et des **limites** assumées (cluster manuscrit exclu, commissions non point-in-time, appariement Quiver sans tolérance de date). Toutes les métriques sont recalculées depuis les tables FINAL ; le document cite, pour chaque mécanisme, la fonction Python réelle qui l'implémente.

Table des matières

Résumé	1
1 Introduction & contexte	3
1.1 Le signal <i>à Congress trading</i> <i>z</i> et le <i>STOCK Act</i>	3
1.2 Ce que Ramify a commandé (Semaines 1–2)	3
1.3 Périmètre et limites de l'étude	3
2 Sources & acquisition	3
3 Architecture & méthodologie	4
3.1 Trois couches de code + une couche de données	4
3.2 Le pipeline : un point d'entrée unique	4
3.3 La convention de numérotation <i>est</i> l'enchaînement	5
3.4 Le voyage d'une transaction	5
4 Le pipeline pas à pas — chaque fonction Python	6
4.1 Cœur partagé — <code>congress_core/</code> (14 modules, 101 fonctions)	6
4.2 Orchestrateur Chambre — <code>house/</code> (3 modules, 48 fonctions)	12
4.3 Orchestrateur Sénat — <code>senate/</code> (10 modules, 79 fonctions)	15
5 Résolution d'identité (priorité #1)	20
6 OCR — lire les scans avec Claude Vision	20
7 Enrichissement : ticker, secteur, montant, ancienneté	21
8 La table finale — schéma et contrat <i>à 12/12 z</i>	21
9 Rapport de qualité & métriques	22
9.1 Les cinq contrôles	22

9.2	Les quatre figures	22
9.3	La couverture par an — et la baisse Sénat 2025–26	22
9.4	Fiabilité des dates OCR (<code>date_confidence</code>)	24
10	Validation externe (Quiver)	24
11	Assurance qualité & reproductibilité	25
12	Hypothèses explorées puis révisées	26
13	Limites (honnêteté)	26
14	Périmètre & suite	27
15	Conclusion	27
16	Annexes	27
16.1	Annexe A — les 44 fichiers <code>.py</code> en un coup d’œil	27
16.2	Annexe B — le schéma FINAL (28 colonnes)	28
16.3	Annexe C — valeurs vérifiées (recalculées depuis les FINAL)	29
16.4	Annexe D — reproduire / lancer	29

1 Introduction & contexte

1.1 Le signal *à Congress trading* et le STOCK Act

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L'intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l'information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (*anti-look-ahead*).

1.2 Ce que Ramify a commandé (Semaines 1–2)

Le projet est structuré en quatre semaines, dont **deux entières dédiées à la donnée** — à un backtest sur des données mal extraites ne vaut rien. La **Semaine 1** couvre l'exploration, le *sourcing* et l'architecture, le téléchargement automatisé, puis la construction du pipeline d'extraction LLM (Claude, Sonnet) avec convention de *midpoint* sur les fourchettes de montant. La **Semaine 2** couvre le nettoyage (tickers, noms incohérents, doublons), la **validation qualité** (cohérence des dates, délai légal, distribution des montants, couverture par membre, achats sans sortie), le pipeline de bout en bout, les métadonnées de sélection (chambre, parti, ancienneté, commissions) et le **mapping sectoriel GICS→ETF**. La contrainte non négociable : une table propre, documentée, où **12 champs** sont garantis présents (cf. §8).

1.3 Périmètre et limites de l'étude

Ce rapport couvre **uniquement la couche données (Semaines 1–2)**. La **stratégie et le backtest (Semaines 3–4)** — règles d'entrée/sortie, sélection annuelle des K congressmen, versions actions puis ETF — constituent un **travail de recherche séparé, en cours**, et ne sont *pas* développés ici. Les limites de la couche données sont énumérées honnêtement au §13.

2 Sources & acquisition

Principe anti-look-ahead. Chaque transaction est datée par sa **date de divulgation** (`disclosure_date`, la date de dépôt du PTR), jamais par une date postérieure connue : une stratégie ne peut copier qu'*après* publication.

Chambre — House Clerk. Un index XML par année (`{an}FD.xml`) liste les dépôts ; on filtre `FilingType=P` (les PTR) et on télécharge le PDF de chaque dépôt. Source publique, sans barrière. Les PDF se répartissent en **lisibles** (couche texte → parsing déterministe) et **scannés** (image → OCR).

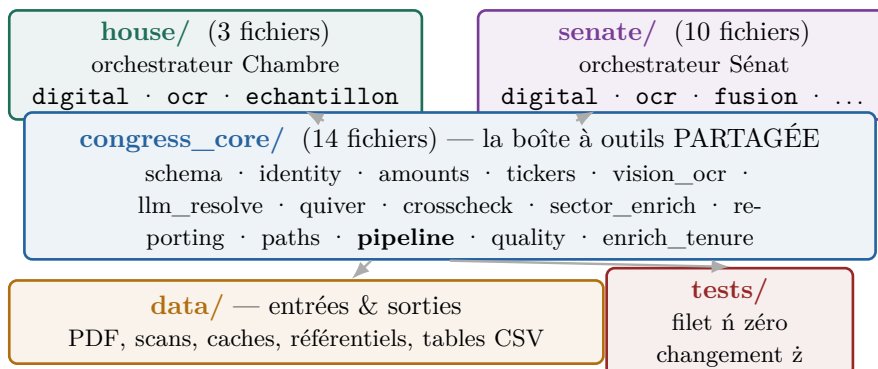
Sénat — eFD (Electronic Financial Disclosure). Plus fermé : un **garde-fou CSRF**. Il faut d'abord accepter un formulaire d'accord (`accept_agreement` : GET pour un `csrftoken`, puis POST `prohibition_agreement` avec `csrftoken` et l'en-tête `X-CSRFToken`). On interroge ensuite l'API de recherche (`report_types=[11]` = PTR). Les déclarations électroniques sont en **HTML** ; les déclarations papier renvoient des **images .gif** servies par `efd-media-public.senate.gov`. On reste à poli (pause entre requêtes, aucun contournement) et on **cache tout sur disque** — un re-run ne re-télécharge rien.

Quiver = vérification externe *uniquement*. Quiver Quantitative (agrégateur commercial) sert de **vérité-terrain indépendante**. Il n'est *jamais* réinjecté dans nos tables : on mesure si notre extraction concorde, on ne la complète pas. C'est une règle structurante du projet.

3 Architecture & méthodologie

3.1 Trois couches de code + une couche de données

La structure est une **boîte à outils partagée** (`congress_core`, 14 modules) que **deux orchestrateurs** (un par chambre) utilisent, plus la donnée et les tests.

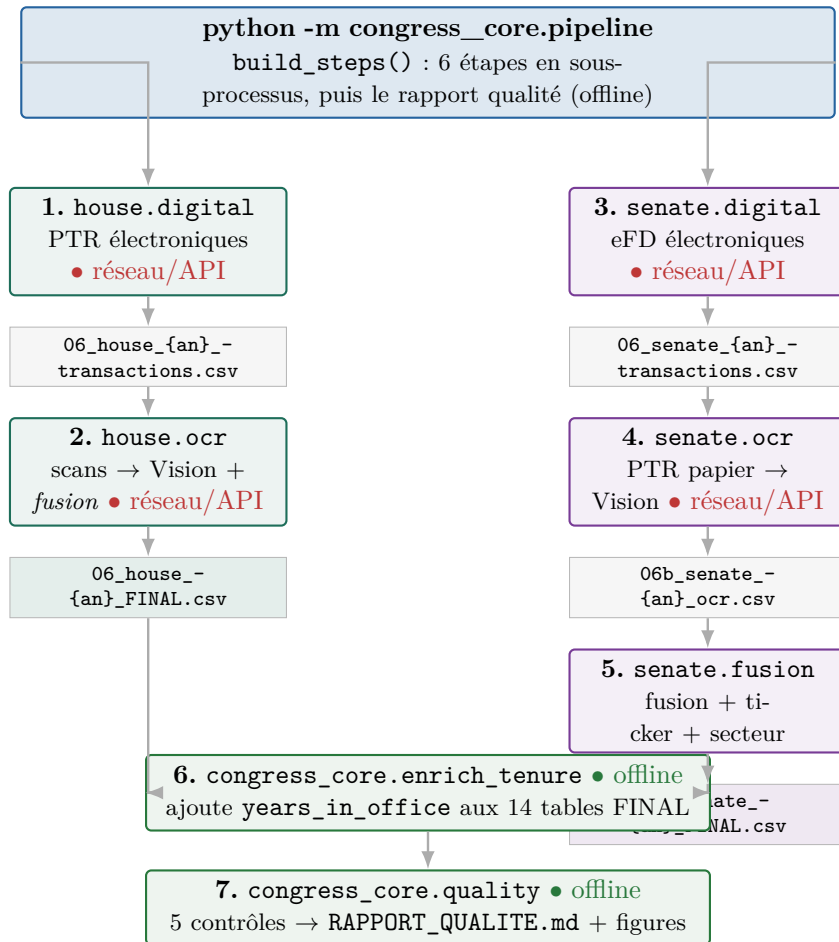


L'idée directrice

`congress_core` = les **ingrédients et ustensiles** (couper un nom en bioguide, lire un scan, calculer un secteur...). `house/` et `senate/` = **deux recettes** qui s'en servent dans leur ordre. Elles ne diffèrent que parce que les **sources** diffèrent : Chambre = index XML + PDF du *Clerk* ; Sénat = eFD (HTML) + images .gif. D'où plus de fichiers côté Sénat (scraping et OCR papier plus complexes).

3.2 Le pipeline : un point d'entrée unique

`congress_core/pipeline.py` (fonction `build_steps`) enchaîne les modules en sous-processus. La commande `python -m congress_core.pipeline -years 2020-2026` assemble **6 étapes** d'orchestration ; un **7^e** traitement — le rapport qualité (`congress_core/quality.py`) — se lance séparément, hors-ligne et en lecture seule.



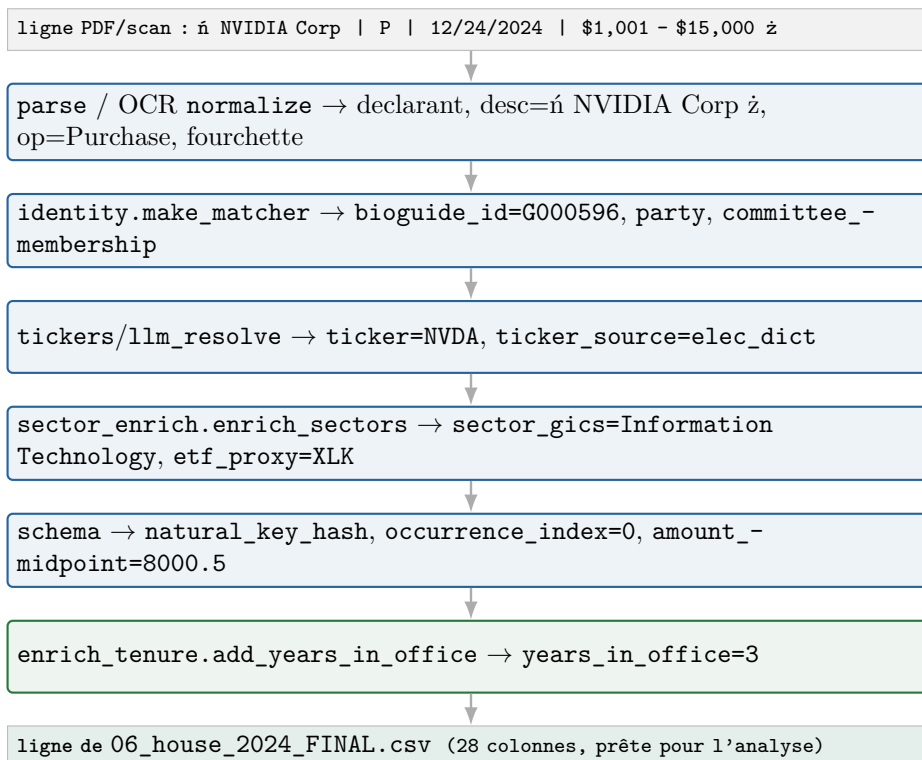
3.3 La convention de numérotation *est* l'enchaînement

Sur le disque, le préfixe d'un fichier = son étape dans le pipeline. C'est ce qui rend la couche données lisible sans documentation externe.

Préfixe	Étape	Écrit par
03_	index des PTR (FilingType=P) dans la fenêtre	house.digital
04_	manifeste : lisible / non_lisible / absent	house.digital
05_	échecs de parsing	*.digital
06_	table digitale (PTR électroniques)	*.digital
06b_	table OCR (scans / papier)	*.ocr
06c_	échecs OCR	*.ocr
06_..._FINAL	digital + OCR fusionnés , 28 colonnes	house.ocr / senate.fusion
07_...07f_	comparaison / réconciliation Quiver (validation)	quiver / quiver_- audit
08_	cross-check à semaine 1 z (Chambre)	house.digital
00_	tableaux de bord (statut, concordance)	*.digital / fusion
_préfixe	caches & brouillons (_quiver*_cache, _scan_census...)	divers

3.4 Le voyage d'une transaction

Comment une ligne brute devient une ligne FINAL — en nommant la fonction à chaque étape.



4 Le pipeline pas à pas — chaque fonction Python

Cette section inventorie **les 228 fonctions et méthodes réelles** des 27 modules de code (les 3 `__init__.py` sans fonction sont omis). Chaque entrée décrit le **corps réel** (pas la docstring), ce que la fonction **renvoie**, et l'**étape du pipeline** où elle intervient. L'inventaire a été dérivé en relisant le code module par module puis recoupé contre l'arbre syntaxique (AST) : *aucune* fonction réelle n'est omise. Pour s'orienter, l'index n étape → où lire z est en Annexe 16.4.

4.1 Cœur partagé — congress_core/ (14 modules, 101 fonctions)

schema.py — Schéma canonique unifié, clé naturelle 7 champs, hash et déduplication non destructrice partagés House/Sénat.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>natural_key</code>	Concatène avec n z les 7 champs (str de chaque), chamber pris de la ligne sinon de l'argument, jamais codé en dur.	<i>str</i> : représentation textuelle stable de la clé	dédup/schéma
<code>natural_key_hash</code>	Encode en UTF-8 la clé naturelle puis calcule son SHA-256 hexadécimal.	<i>str</i> : empreinte SHA-256 hexadécimale	dédup/schéma
<code>add_occurrence_index</code>	Copie le DataFrame, ajoute <code>occurrence_index</code> = rang cumcount par groupe (<code>doc_id</code> , <code>natural_key_hash</code>) pour préserver les lots répétés.	<i>DataFrame</i> avec colonne <code>occurrence_index</code>	dédup/schéma
<code>dedup_canonical</code>	Trie par <code>disclosure_date</code> décroissant puis garde une ligne par (<code>natural_key_hash</code> , <code>occurrence_index</code>), retirant les doublons cross-dépôt.	<i>DataFrame</i> déduplicé canonique	dédup/schéma

identity.py — Identité : normalisation de noms, référentiel des législateurs/commissions, résolution bioguide et ancienneté.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>strip_accents</code>	Normalise en NFKD et retire les caractères combinants pour ôter les accents d'une chaîne.	<i>str</i> : chaîne sans accents	identité
<code>norm</code>	Retire accents, passe en minuscules, remplace tout non-alphabétique par des espaces, réduit les espaces.	<i>str</i> : nom normalisé	identité
<code>split_name</code>	Découpe <code>Prénom ... Nom[suffixe]</code> <code>z</code> : prend la partie avant la virgule, ôte les suffixes, renvoie dernier, premier et milieux.	<i>tuple</i> (<i>last</i> , <i>first</i> , <i>liste des milieux</i>)	identité
<code>Reference.__init__</code>	Stocke les structures du référentiel : univers, index de noms, commissions, <code>key_bios</code> , bios courants, source, première année.	— (<i>effet de bord</i> : initialise l'objet)	identité
<code>Reference.party</code>	Renvoie le parti du bioguide depuis <code>ref_universe</code> si présent dans l'index, sinon <code>None</code> .	<i>str</i> parti ou <code>None</code>	identité
<code>Reference.years_in_office</code>	Calcule <code>as_of_year</code> moins la première année en fonction ; <code>None</code> si données manquantes ou résultat négatif.	<i>int</i> ancienneté ou <code>None</code>	ancienneté
<code>Reference.committees</code>	Joint par <code>z</code> les commissions triées du bioguide, ou chaîne vide si bioguide absent.	<i>str</i> : commissions ou vide	identité
<code>Reference.is_key_committee</code>	Teste l'appartenance du bioguide à l'ensemble <code>key_bios</code> ; <code>False</code> si bioguide vide.	<i>bool</i> : membre d'une commission clé	identité
<code>_last_term_key</code>	Extrait l'année de fin du dernier mandat et la renvoie négative pour tri ; valeurs spéciales si absente ou illisible.	<i>int</i> : clé de tri par fin de mandat	utilitaire
<code>_load_people</code>	Tente de télécharger législateurs current et historical en live, repli sur YAML embarqué en cas d'échec.	<i>tuple</i> (<i>cur</i> , <i>hist</i> , <i>source</i>)	acquisition
<code>_load_committees</code>	Télécharge commissions et appartenances en live, repli sur YAML embarqué si échec ou non-live.	<i>tuple</i> (<i>committees</i> , <i>membership</i>)	acquisition
<code>load_reference</code>	Construit le référentiel : trie les personnes, indexe variantes de noms (surnom, milieu, composé), première année, commissions, <code>key_bios</code> .	<i>objet Reference</i> assemblé	identité
<code>make_matcher</code>	Fabrique une closure <code>match(last, first)</code> : override manuel, candidats nom/prénom avec surnoms, lookup exact, repli nom unique.	<i>fonction</i> <code>match(last, first) → bioguide ou None</code>	identité
<code>enrich_identity</code>	Copie le DataFrame, mappe bioguide via <code>matcher</code> (colonnes <code>last/first</code> ou <code>split_name</code>), ajoute <code>chamber</code> , <code>party</code> , commissions, <code>key_flag</code> .	<i>DataFrame</i> enrichi en identité	identité
<code>add_years_in_office</code>	Copie le DataFrame, extrait l'année de transaction (repli divulgation) et calcule <code>years_in_office</code> par bioguide.	<i>DataFrame</i> avec colonne <code>years_in_office</code>	ancienneté

amounts.py — Fourchettes de montant, *midpoint*, propriétaire et type d'opération, maps indexées par chambre sans fusion.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>amount_midpoint</code>	Extrait les nombres <code>\$\$\$</code> de la chaîne : moyenne des deux premiers, sinon le seul nombre, sinon <code>None</code> .	<i>float</i> <i>midpoint</i> ou <code>None</code>	parsing montant
<code>operation_type_from_code</code>	Mappe un code en majuscules : <code>P→Purchase</code> , <code>E→Exchange</code> , sinon <code>Sale</code> avec variante <code>Partial/Full</code> selon le sous-type.	<i>str</i> : type d'opération 5-voies	parsing

tickers.py — Normalisation et récupération de ticker, inférence du type d'actif par chambre, copies exactes des moteurs.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>recover_ticker</code>	Si la description correspond au motif <code>le nom EST</code> un ticker <code>z</code> , renvoie le symbole capté, sinon <code>None</code> .	<i>str</i> ticker ou <code>None</code>	ticker

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>explicit_ticker</code>	Cherche un symbole explicite en fin de description, ôte les points, renvoie s'il fait 1 à 5 caractères.	<i>str ticker ou None</i>	ticker
<code>norm_asset</code>	Met en majuscules, retire ticker explicite et ponctuation, supprime les suffixes corporatifs, applique des corrections OCR.	<i>str : description normalisée pour lookup</i>	ticker
<code>_infer_asset_type_house</code>	Teste en cascade des regex House (gouvernemental, structuré, fonds, obligation, action) sur la description majuscule.	<i>str type d'actif ou None</i>	secteur
<code>_infer_asset_type_senate</code>	Teste en cascade des regex Sénat (municipal, obligation, autre, action) sur la description majuscule.	<i>str type d'actif ou None</i>	secteur
<code>infer_asset_type</code>	Aiguille vers la variante Sénat ou House selon le paramètre <code>chamber</code> pour inférer le type d'actif.	<i>str type d'actif ou None</i>	secteur

vision_ocr.py — Moteur OCR Claude Vision chambre-agnostique : rendu image, deskew, appel `tool_use` et cache versionné avec reprise.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>OcrError</code>	Classe d'exception dédiée aux échecs OCR ; corps vide (juste pass).	— (<i>classe d'exception, pas de return</i>)	utilitaire
<code>rotate_b64_png</code>	Décode le PNG b64, le tourne de -angle via PIL (horaire), réencode ; retourne tel quel si multiple de 360.	<i>str : PNG base64 tourné</i>	OCR Vision (deskew)
<code>detect_rotation</code>	Montre les 4 rotations à Vision, lit le chiffre choisi par regex, retourne l'angle ; backoff puis 0 si échec.	<i>int : angle horaire (0 par défaut)</i>	OCR Vision (deskew)
<code>pdf_to_b64_images</code>	Rend chaque page PDF en PNG b64 via <code>pymupdf</code> ; si deskew, détecte et redresse chaque page en basse résolution.	<i>liste PNG b64, ou (images, angles) si deskew</i>	OCR Vision (rendu)
<code>image_b64_from_url</code>	Télécharge l'image (cache disque clé SHA1 de l'URL), la redimensionne sous <code>long_edge</code> , renvoie PNG b64.	<i>str : PNG base64 redimensionné</i>	OCR Vision (rendu)
<code>VisionExtractor.__init__</code>	Stocke modèle, prompt, tool, <code>pipeline_tag</code> et paramètres (<code>max_tokens</code> , images par appel) sur l'instance.	— (<i>effet de bord : initialise l'objet</i>)	OCR Vision (init)
<code>VisionExtractor.prompt_sha</code>	Propriété : calcule le SHA256 tronqué à 12 de prompt plus tool trié plus <code>pipeline_tag</code> pour versionner le cache.	<i>str : empreinte hex de 12 caractères</i>	OCR Vision (versionnage)
<code>VisionExtractor.call</code>	Un appel Vision avec <code>tool_use</code> forcé, extrait les transactions du bloc <code>tool_use</code> ; backoff sur 429/5xx jusqu'à <code>max_retries</code> .	<i>list : transactions extraites</i>	OCR Vision (appel API)
<code>VisionExtractor.extract_cached</code>	Lit le cache si version concorde, sinon batche les images et n'appelle que les batches manquants ou en erreur, puis écrit le JSON.	<i>(liste transactions, dict objet cache)</i>	OCR Vision (cache, reprise)

llm_resolve.py — Résolveur LLM générique à cache versionné ; factorise le motif `tool_use` et porte la passe House nom vers ticker.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>VersionedLLMCache._init__</code>	Stocke le chemin, le modèle et le <code>prompt_sha</code> qui versionnent le cache disque.	— (<i>effet de bord : initialise l'objet</i>)	ticker (cache)
<code>VersionedLLMCache.load</code>	Lit le JSON disque ; renvoie les mappings seulement si modèle et <code>prompt_sha</code> concordent, sinon dictionnaire vide.	<i>dict : mappings, ou {} si version différente</i>	ticker (cache)

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>VersionedLLMCache.save</code>	Écrit sur disque un JSON contenant modèle, <code>prompt_sha</code> et les mappings fournis.	— (<i>effet de bord : écrit le fichier cache</i>)	ticker (cache)
<code>map_batch</code>	Un appel <code>tool_use</code> forcé renvoyant l'input du bloc <code>tool_use</code> ; backoff sur 429/5xx, relève après <code>max_retries</code> .	<i>dict</i> : <i>input du tool</i> , ou <i>{}</i> si absent	ticker (appel LLM)
<code>ticker_prompt_sha</code>	Calcule le SHA256 tronqué à 16 de <code>TICKER_PROMPT</code> plus <code>TICKER_TOOL</code> trié plus modèle pour versionner le cache.	<i>str</i> : <i>empreinte hex de 16 caractères</i>	ticker (versionnage)
<code>resolve_names_to_tickers</code>	Pour les lignes sans ticker, interroge le LLM par lots (cache versionné), valide via <code>regex/is_equity</code> , puis remplit ticker et marque la source <code>llm</code> .	<i>DataFrame</i> : <i>df avec tickers complétés</i>	ticker (résolution)

quiver.py — Vérification externe Quiver (jamais réinjectée) : *fetch cache-first*, clés d'appariement et réconciliation House et Sénat.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>norm_ticker</code>	Met en majuscules, vide les valeurs nulles, retire suffixe PUT/CALL, remplace points et tirets par underscore.	<i>str</i> : <i>ticker normalisé (vide si invalide)</i>	validation Quiver (normalisation)
<code>norm_sense</code>	Réduit le sens à 3 valeurs selon la première lettre : p->Purchase, s->Sale, e->Exchange, sinon brut ou point d'interrogation.	<i>str</i> : <i>sens canonique 3-voies</i>	validation Quiver (normalisation)
<code>low_bound</code>	Extrait par regex le premier montant en dollars de la fourchette et le convertit en entier sans virgules.	<i>int</i> : <i>borne basse, ou None si absent</i>	validation Quiver (montant)
<code>fetch_quiver</code>	Charge le cache CSV s'il existe, sinon appelle l'API avec le token, filtre par chambre et renomme les colonnes; None si pas de token.	<i>DataFrame</i> <i>Quiver filtré, ou None</i>	validation Quiver (fetch)
<code>validate_quiver_house</code>	Filtre Quiver sur la fenêtre, compare les comptes par déclarant brut, recoupe au niveau transaction par clé brute, calcule verdicts et manquants réels.	(<i>cmp_df</i> , <i>real_missing_df</i> , <i>stats dict</i>)	validation Quiver (House)
<code>reconcile</code>	Dédoublonne Quiver, apparie par clé <code>bioguide×ticker×date×sens</code> , mesure accords <code>sens/date/montant</code> et <code>deltas</code> par membre.	<i>dict de DataFrames (reco, accords, deltas)</i>	validation Quiver (Sénat)

crosscheck.py — Triangulation multi-sources et statut de validation par déposant; matérialise que le papier OCR est source unique.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_norm_name</code>	Normalise un nom : minuscules, ne garde que lettres et espaces, compresse les espaces multiples.	<i>str</i> : <i>nom normalisé pour appariement</i>	utilitaire (normalisation nom)
<code>per_filer_status</code>	Agrège par bioguide nos comptes total/OCR/digital, joint Quiver, puis attribue un statut (<code>quiver_validable</code> , <code>ocr_unique</code> , <code>digital</code>).	<i>DataFrame</i> : <i>statut trié par volume</i>	qualité (statut déposant)
<code>load_kadoa_house</code>	Lit le JSON Kadoa, garde les entrées chambre house, mappe nom normalisé vers <code>trade_count</code> .	<i>dict</i> : <i>nom normalisé -> nombre de trades</i>	triangulation (Kadoa)
<code>load_hsw_counts</code>	Lit le JSON House Stock Watcher et compte les transactions par représentant (nom normalisé).	<i>dict</i> : <i>nom normalisé -> nombre de transactions</i>	triangulation (HSW)
<code>add_external_counts</code>	Joint par nom normalisé les comptes Kadoa et HSW au tableau de statut comme colonnes informatives.	<i>DataFrame</i> : <i>statut enrichi des comptes externes</i>	triangulation (fusion externe)

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
summary	Regroupe le tableau de statut par statut et somme déposants, transactions et part OCR.	<i>DataFrame</i> : <i>récapitulatif par statut</i>	qualité (récapitulatif)

sector_enrich.py — *Enrichissement secteur : ticker → GICS → ETF SPDR, hybride yfinance + repli/audit LLM + overrides manuels.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
load_cache	Lit le JSON de cache du chemin ; renvoie les mappings seulement si pipeline_sha et model concordent, sinon dict vide.	<i>dict des mappings ticker→infos (vide si invalide)</i>	ticker/cache
save_cache	Écrit le cache JSON (model, pipeline_sha, mappings) au chemin donné, indentation 1, sans échappement Unicode.	— (effet de bord : écrit le fichier cache)	ticker/cache
_yf_sector_one	Importe yfinance, lit le champ sector d'un ticker, le traduit en GICS via la table ; None si erreur ou absent.	<i>str secteur GICS ou None</i>	secteur (yfinance)
resolve_yfinance	Interroge yfinance ticker par ticker sur plusieurs passes, ré-essaye les None avec pauses croissantes.	<i>dict ticker→secteur GICS (ou None)</i>	secteur (yfinance)
_map_sectors_batch	Appelle l'API Claude avec outil forcé map_sectors, ré-essaie avec back-off sur erreurs réseau ; extrait les mappings.	<i>liste de mappings (dicts ticker/secteur)</i>	secteur (LLM)
resolve_llm	Sans clé API renvoie vide ; sinon batche les tickers vers le LLM, ne garde le secteur que si coté et valide.	<i>dict ticker→secteur GICS validé ou None</i>	secteur (LLM)
_norm_ticker	Convertit en chaîne, retire les espaces et met en majuscules.	<i>str ticker normalisé</i>	utilitaire
resolve_sectors	Filtre/normalise les tickers valides, calcule les manquants, croise yfinance puis LLM, choisit le final, sauvegarde le cache.	<i>dict cache complet ticker→infos secteur</i>	secteur (orchestration)
enrich_sectors	Copie le df, résout les secteurs, ajoute colonnes sector_gics/etf_proxy/sector_source, applique les overrides manuels par ticker.	<i>DataFrame enrichi (3 colonnes ajoutées)</i>	secteur (enrichissement)
build_audit	Pour chaque ticker du df présent au cache, construit une ligne d'audit yfinance vs LLM avec drapeau d'accord.	<i>DataFrame d'audit secteur croisé</i>	secteur (audit)

reporting.py — *Sorties partagées : convention de nommage des tables, drapeaux QA, dashboard idempotent, export Excel multi-onglets.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
qa_flags	Parcourt chaque ligne, repère les champs obligatoires vides ou n'ni nan z, accumule une ligne d'anomalie listant ces champs.	<i>DataFrame des anomalies (champs manquants)</i>	qualité
upsert_status	Construit un DataFrame, relit le CSV existant, retire les lignes de même clé puis concatène et réécrit (idempotent).	<i>DataFrame du dashboard mis à jour</i>	dashboard
write_excel	Ouvre un classeur openpyxl, écrit un onglet LISEZMOI optionnel puis un onglet par DataFrame (placeholder si vide).	— (effet de bord : écrit le fichier Excel)	sorties (Excel)

paths.py — *Bootstrap des chemins et secrets ; DataRoot route les I/O sous data/{chambre}/ par chambre.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_env</code>	Charge le <code>.env</code> du <code>base_dir</code> ; si <code>ANTHROPIC_API_KEY</code> manque encore, charge un <code>.env</code> trouvé ailleurs dans l'arbre.	— (effet de bord : variables d'environnement chargées)	utilitaire (secrets)
<code>get_secret</code>	Renvoie la variable d'environnement ; sinon parse ligne à ligne le <code>.env</code> local pour la clé demandée.	<i>str</i> valeur du secret ou <i>None</i>	utilitaire (secrets)
<code>DataRoot.__init__</code>	Stocke le nom de chambre et le <code>data_dir</code> converti en <code>Path</code> .	— (effet de bord : initialise l'instance)	utilitaire (chemins)
<code>DataRoot.for_chamber</code>	Fabrique de classe : construit un <code>DataRoot</code> pointant sur <code>repo_root/data/{chambre}</code> .	instance <i>DataRoot</i>	utilitaire (chemins)
<code>DataRoot.tables</code>	Propriété renvoyant le sous-dossier <code>tables</code> de la chambre.	<i>Path</i> du dossier <i>tables</i>	utilitaire (chemins)
<code>DataRoot.tables_dir</code>	Renvoie <code>tables/{année}</code> si une année est fournie, sinon le dossier <code>tables</code> racine.	<i>Path</i> du dossier <i>tables</i> (par année)	utilitaire (chemins)
<code>DataRoot.pdfs</code>	Propriété renvoyant le sous-dossier <code>pdfs</code> de la chambre.	<i>Path</i> du dossier <i>pdfs</i>	utilitaire (chemins)
<code>DataRoot.index</code>	Propriété renvoyant le sous-dossier <code>index</code> de la chambre.	<i>Path</i> du dossier <i>index</i>	utilitaire (chemins)
<code>DataRoot.reference</code>	Propriété renvoyant le sous-dossier <code>reference</code> de la chambre.	<i>Path</i> du dossier <i>reference</i>	utilitaire (chemins)
<code>DataRoot.ocr_cache</code>	Propriété renvoyant le sous-dossier <code>ocr_cache</code> de la chambre.	<i>Path</i> du dossier <i>ocr_cache</i>	utilitaire (chemins)
<code>DataRoot.reports</code>	Propriété renvoyant le sous-dossier <code>reports</code> de la chambre.	<i>Path</i> du dossier <i>reports</i>	utilitaire (chemins)
<code>DataRoot.media</code>	Propriété renvoyant le sous-dossier <code>media</code> sous <code>reports</code> .	<i>Path</i> du dossier <i>media</i>	utilitaire (chemins)
<code>DataRoot.quiver_cache_path</code>	Propriété construisant le chemin du CSV de cache <code>Quiver</code> propre à la chambre sous <code>tables</code> .	<i>Path</i> du fichier <i>cache Quiver</i>	utilitaire (chemins)

pipeline.py — *Pipeline end-to-end unifié : orchestre par sous-processus les étapes digital/OCR/fusion des deux chambres.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>parse_years</code>	Parse la spec d'années : intervalle <code>á a-b z</code> en range, sinon liste <code>á x,y z</code> ou année unique en entiers.	<i>list[int]</i> des années	orchestration (CLI)
<code>build_steps</code>	Assemble la séquence ordonnée (<code>label, args-module</code>) selon <code>skip_ocr/force/no_quiver/mode</code> , ajoute l'enrichissement ancienneté final.	<i>list</i> de tuples (<i>label, args-module</i>)	orchestration
<code>main</code>	Parse les arguments CLI, construit les étapes, les imprime puis lance chaque module en sous-processus, arrêt au premier échec.	— (effet de bord : exécute le pipeline, <i>sys.exit</i> si échec)	orchestration

quality.py — *Rapport qualité Ramify : charge les FINAL, calcule cinq contrôles, génère figures et RAPPORT_QUALITE.md.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_final_path</code>	Construit le chemin de la table <code>FINAL</code> selon la chambre (<code>House</code> sous <code>tables/{année}</code> , <code>Sénat</code> sous <code>{année}</code>).	<i>Path</i> de la table <i>FINAL</i>	chargement (chemins)
<code>op_class</code>	Met en minuscules l'opération <code>_type</code> et classe en <code>buy/sell/exchange/other</code> selon les mots-clés présents.	<i>str</i> classe canonique d'opération	qualité (normalisation)
<code>load_final</code>	Concatène tous les <code>FINAL</code> des deux chambres, ajoute <code>file_year/txn_year</code> , parse les dates, calcule <code>lag_days</code> , <code>midpoint</code> et <code>op</code> .	<i>DataFrame</i> consolidé enrichi	chargement/fusion

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>date_coherence</code>	Par chambre : compte dates parseables, part cohérente ($\text{lag} \geq 0$), incohérentes, années de transaction implausibles, dates manquantes.	<i>DataFrame résumé de cohérence des dates</i>	qualité (a) dates
<code>delay_buckets</code>	Par chambre : ventile le délai de divulgation en tranches $\leq 45j/45-75j/>75j$ /négatif et la médiane positive.	<i>DataFrame ventilation des délais</i>	qualité (b) délai
<code>delay_outliers</code>	Filtre les délais au-dessus du seuil, trie décroissant, garde le top et un sous-ensemble de colonnes.	<i>DataFrame des divulgations les plus tardives</i>	qualité (b) délai
<code>amount_distribution</code>	Calcule stats descriptives globales et par chambre du midpoint, plus le top 15 déposants par volume estimé.	<i>dict {overall, by_chamber, top_volume}</i>	qualité (c) montants
<code>coverage_per_member</code>	Appelle <code>crosscheck.per_filer_status</code> par chambre, fusionne nombre d'années actives et première/dernière année de transaction.	<i>DataFrame de couverture par membre</i>	qualité (d) couverture
<code>eligible_members</code>	Filtre les membres avec au moins <code>min_trades</code> transactions ; compte éligibles, total et éligibles actifs ≥ 3 ans.	<i>dict de comptes d'éligibilité</i>	qualité (d) couverture
<code>unmatched_purchase_rate</code>	Apparie chaque achat (bioguide+ticker) à une vente ultérieure ; calcule par chambre la part d'achats sans sortie déclarée.	<i>DataFrame du taux d'achats non appariés</i>	qualité (e) sorties
<code>_figures</code>	Avec <code>matplotlib</code> Agg, génère quatre PNG (délai, montants log, transactions par an, top déposants) dans le dossier.	<i>list des chemins de figures écrites</i>	qualité (figures)
<code>_md_table</code>	Formate un <code>DataFrame</code> en table Markdown manuelle, échappant les barres et vidant les valeurs nulles.	<i>str table Markdown</i>	rapport (Markdown)
<code>build_report</code>	Charge les données, calcule les cinq contrôles et figures, assemble le Markdown puis écrit <code>RAPPORT_QUALITE.md</code> .	<i>Path du rapport Markdown écrit</i>	rapport (Markdown)
<code>main</code>	Ancre le <code>repo_root</code> , appelle <code>build_report</code> et imprime les chemins du rapport et des figures.	— (effet de bord : génère le rapport)	orchestration

enrich_tenure.py — *Passe post-FINAL ajoutant `years_in_office` aux deux chambres, offline et byte-à-byte préservante.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>final_files</code>	Génère (chambre, année, chemin) pour chaque table FINAL existante, House sous tables/ puis Sénat sous {année}.	<i>itérateur de tuples (chambre, année, Path)</i>	ancienneté (découverte)
<code>_fmt</code>	Renvoie chaîne vide si None ou NaN, sinon l'entier converti en chaîne.	<i>str valeur formatée</i>	utilitaire
<code>enrich_file</code>	Lit le brut, saute si colonne déjà présente, vérifie la cohérence lignes/enregistrements, append <code>years_in_office</code> en préservant les octets.	<i>bool True si enrichi, False si déjà fait</i>	ancienneté (enrichissement)
<code>main</code>	Pour chaque FINAL, charge la référence de chambre (mise en cache), enrichit le fichier et imprime un récapitulatif des comptes.	— (effet de bord : enrichit les fichiers FINAL)	orchestration

4.2 Orchestrateur Chambre — `house/` (3 modules, 48 fonctions)

digital.py — *Pipeline piste digitale House multi-années : parse PTR lisibles, identité, dédup, cross-check baseline et validation Quiver.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
load_dotenv	Repli défini seulement si python-dotenv absent : ne fait rien et renvoie False.	<i>False (stub si dotenv manquant)</i>	utilitaire
build_reference	Charge le référentiel via congress_core.identity et peuple les globals identité (univers, index noms, comités, matcher).	— (<i>effet de bord : peuple globals, imprime</i>)	identité
_find_continuation	Cherche un code actif [XX] suivi d'un montant dans une ligne ; reformate un préfixe tiret en parenthèses.	<i>tuple (code_actif, second_montant, préfixe)</i>	parsing
_join_split_lines	Recolle les transactions éclatées sur 2 ou 3 lignes en réinsérant code actif et second montant au bon endroit.	<i>liste de lignes recollées</i>	parsing
_is_txn_start	Vérifie qu'une ligne commence une transaction : motif TXN au début ou précédé d'un code actif.	<i>booléen (début de transaction)</i>	parsing
parse_ptr	Parse le texte PTR : recolle lignes, détecte transactions, agrège blocs descriptifs, extrait ticker, type actif, propriétaire, montant.	<i>liste de dicts transactions</i>	parsing
match_bioguide	Délègue au matcher du cœur peuplé par build_reference ; renvoie None si matcher absent.	<i>identifiant bioguide ou None</i>	identité
load_ptr_index	Lit l'index XML de l'année, construit un DataFrame, filtre FilingType=P dans la fenêtre de divulgation, ajoute l'URL PDF.	<i>(DataFrame PTR filtré, distribution filing_type)</i>	parsing
resolve_pdf_path	Construit le chemin du PDF par année et doc_id ; renvoie le Path s'il existe, sinon None.	<i>Path du PDF ou None</i>	utilitaire
extract_text	Ouvre le PDF avec pdfplumber et concatène le texte de toutes les pages ; renvoie vide en cas d'erreur.	<i>texte du PDF (str)</i>	parsing
n_pages	Ouvre le PDF avec pdfplumber et renvoie le nombre de pages ; None en cas d'erreur.	<i>nombre de pages ou None</i>	utilitaire
build_manifest	Lit chaque PDF en place, extrait le texte, route en lisible/non_lisible/absent, écrit le manifeste CSV.	<i>(textes par doc, DataFrame manifeste)</i>	OCR routage
parse_docs	Parse chaque texte lisible, accumule lignes et échecs, écrit les échecs CSV, calcule le rendement.	<i>(lignes, échecs, nb lisibles, taux rendement)</i>	parsing
join_identity	Fusionne lignes parsées avec l'index, résout bioguide, mappe parti, comités, drapeau commission clé, chambre.	<i>DataFrame enrichi identité</i>	identité
_legacy_key	Délègue au cœur partagé pour calculer le hash de clé naturelle House d'une ligne.	<i>hash de clé naturelle (str)</i>	dédup/schéma
finalize	Normalise champs, calcule clé naturelle et indice d'occurrence, déduplique en gardant la divulgation la plus récente.	<i>(table canonique, table complète, stats dédup)</i>	dédup/schéma
_sem1_year	Charge en cache la baseline historique et filtre les lignes dont l'année de divulgation égale l'année demandée.	<i>DataFrame baseline filtré par année</i>	validation baseline
crosscheck_semaine1	Compare comptes par doc entre nous et la baseline, détecte dérives (baseline>0 et nous=0), écrit le CSV.	<i>dict métriques de cross-check</i>	validation baseline
fetch_quiver	Charge le cache Quiver House si présent, sinon appelle l'API live avec token, filtre la chambre, met en cache.	<i>DataFrame Quiver ou None</i>	validation Quiver
validate_quiver	Compare comptes par déclarant per-lot et recouvrement au niveau transaction vs Quiver, distingue manques papier, écrit CSV.	<i>dict métriques de validation ou None</i>	validation Quiver
run_year	Orchestre une année : index, manifeste, parsing, identité, finalisation, cross-check, Quiver, backlog OCR ; écrit fichiers.	<i>(résumé, table finale, backlog)</i>	orchestration
main	Analyse les arguments, construit le référentiel, lance run_year par année, fusionne et écrit statut et backlog cumulés.	— (<i>effet de bord : écrit CSV, imprime</i>)	orchestration

ocr.py — OCR multi-années des PTR scannés via Claude Vision : deskew, extraction outillée, cache, ticker LLM, fusion finale et validation Quiver.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_load_ticker_cache</code>	Lit le cache JSON ticker LLM ; ne renvoie les mappings que si <code>prompt_sha</code> et modèle concordent.	<i>dict des mappings</i> <i>nom→ticker</i>	ticker
<code>_save_ticker_cache</code>	Écrit le cache ticker LLM en JSON avec modèle, <code>prompt_sha</code> et mappings.	— (effet de bord : écrit le cache JSON)	ticker
<code>_map_tickers_batch</code>	Appelle l'API outil <code>map_tickers</code> sur un lot de noms avec retries à backoff ; extrait les mappings du bloc <code>tool_use</code> .	<i>liste de mappings</i> ou <i>exception</i>	ticker
<code>llm_resolve_tickers</code>	Pour les lignes sans ticker, interroge le LLM par lots de 60, met en cache, remplit ticker et marque la source llm.	<i>DataFrame</i> avec <i>tickers complétés</i>	ticker
<code>detect_rotation</code>	Montre la page aux 4 rotations au modèle, lui fait choisir celle droite, renvoie l'angle horaire ; 0 si échec.	<i>angle horaire</i> à <i>appliquer (int)</i>	OCR Vision
<code>rotate_b64_png</code>	Décode le PNG base64 et le tourne de l'angle horaire (PIL anti-horaire donc négatif) ; ré-encode en base64.	<i>PNG base64 tourné</i> <i>(str)</i>	OCR Vision
<code>pdf_to_b64_images</code>	Rend chaque page en PNG base64 ; si deskew détecte et redresse page par page, renvoie images et angles.	<i>images</i> ou (<i>images</i> , <i>angles</i>)	OCR Vision
<code>OcrError</code>	Classe d'exception dédiée aux erreurs d'extraction OCR ; corps vide.	— (<i>type</i> <i>d'exception</i>)	utilitaire
<code>_call_vision_tool</code>	Envoie les images et le prompt à Claude avec l'outil <code>record_transactions</code> , retries selon code HTTP ; extrait les transactions.	<i>liste de transactions</i> ou <i>OcrError</i>	OCR Vision
<code>extract_from_pdf</code>	Extraction avec cache versionné et reprise par lot : ne refait que les lots en erreur, écrit le cache JSON.	(<i>transactions</i> , <i>objet</i> <i>cache</i>)	OCR Vision
<code>natural_key_hash</code>	Délègue au cœur partagé pour calculer le hash de clé naturelle House d'une ligne.	<i>hash de clé naturelle</i> <i>(str)</i>	dédup/schéma
<code>date_confidence</code>	Calcule le délai dépôt-transaction ; renvoie plausible si entre 0 et 75 jours, sinon implausible.	' <i>plausible</i> ' ou ' <i>implausible</i> ' (<i>str</i>)	qualité
<code>normalize</code>	Mappe code montant, propriétaire, dates ; construit la ligne au schéma, calcule confiance de date et clé naturelle.	<i>dict ligne</i> <i>normalisée</i>	dédup/schéma
<code>_infer_asset_type</code>	Délègue au cœur l'inférence du type d'actif depuis la description, contexte House.	<i>type d'actif (str</i> ou <i>None)</i>	secteur
<code>run_ocr_year</code>	Orchestre l'OCR d'une année : sélectionne scannés (exclut cluster C sauf filers prioritaires), extrait en parallèle, enrichit, fusionne finale.	<i>dict résumé</i> ou <i>None</i>	orchestration
<code>validate_ocr_quiver</code>	Compare comptes OCR vs Quiver par déclarant fenêtré par année, attribue un verdict, écrit le CSV.	<i>dict décompte des</i> <i>verdicts</i> ou <i>None</i>	validation Quiver
<code>main</code>	Analyse arguments, vérifie la clé API, construit le référentiel, lance OCR par année puis validation Quiver.	— (effet de bord : <i>imprime, écrit</i> <i>fichiers</i>)	orchestration

echantillon.py — OCR sur échantillon représentatif par année (~70 docs) : prompt durci, deskew, enrichissement ticker et comparaison Quiver par cluster et année.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_route</code>	Choisit DPI et pages-par-appel selon la densité : doc dense renvoie DPI haut et 1 page, sinon DPI moyen et 3.	<i>tuple (dpi</i> , <i>pages_par_appel)</i>	OCR Vision
<code>_render</code>	Rend les pages (plafonnées au cap) en PNG base64 ; avec client détecte et redresse l'orientation page par page.	(<i>images</i> , <i>angles</i> <i>appliqués</i>)	OCR Vision

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_call</code>	Envoie les images et le prompt durci à Claude avec l'outil <code>record_transactions</code> , retries à backoff ; extrait les transactions.	<i>liste de transactions</i>	OCR Vision
<code>_ocr_doc</code>	OCR d'un doc échantillon : sert le cache durci versionné sinon ré-OCR redressé ; ne lève jamais, renvoie liste vide si échec.	<i>(doc_id, transactions)</i>	OCR Vision
<code>_global_ticker_dict</code>	Agrège toutes les tables digitales 2020-2026 en dictionnaire nom normalisé→ticker (première valeur retenue).	<i>dict nom normalisé→ticker</i>	ticker
<code>run_echantillon</code>	Orchestre l'OCR de l'échantillon (exclut cluster C), normalise, enrichit ticker et bioguide, écrit les résultats CSV.	<i>DataFrame des transactions OCR</i>	orchestration
<code>_nearest_signed</code>	Trouve la date Quiver la plus proche d'une date donnée et calcule l'écart signé en jours.	<i>(date proche, écart signé) ou (None, None)</i>	validation Quiver
<code>compare_quiver</code>	Calcule par doc la précision ticker et ticker+date (tolérance date) contre Quiver fenêtré ; écrit le CSV par doc.	<i>DataFrame précision par doc</i>	validation Quiver
<code>detailed_quiver_diff</code>	Diff transaction par transaction vs Quiver : statut match, date fausse, ticker faux, sans données ou sans ticker ; écrit CSV.	<i>DataFrame diff par transaction</i>	validation Quiver

4.3 Orchestrateur Sénat — `senate/` (10 modules, 79 fonctions)

digital.py — Pipeline Sénat digital multi-années : scraping eFD des PTR électroniques, parsing, enrichissement, validation Quiver par an.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>accept_agreement</code>	GET la page d'accueil eFD, lit le csrftoken, POST l'agrément de prohibition avec le token.	<i>bool : sessionid ou csrftoken présent dans les cookies</i>	acquisition
<code>fetch_ptr_list</code>	POST paginé l'endpoint data eFD (<code>report_types[11]</code>), extrait nom/kind/uuid/date par regex, filtre sur la fenêtre.	<i>DataFrame des dépôts PTR de la fenêtre</i>	acquisition
<code>fetch_report</code>	Renvoie le HTML du PTR depuis le cache disque s'il existe, sinon GET, écrit le cache, pause.	<i>(url, texte HTML) ou (url, chaîne vide)</i>	acquisition
<code>amount_midpoint</code>	Extrait les nombres dollar par regex ; moyenne des deux premiers, sinon valeur unique, sinon None.	<i>float milieu de fourchette, ou None</i>	parsing
<code>norm_op</code>	Normalise le type d'opération par préfixe/mot-clé : Purchase, Exchange, Sale (Partial/Full), Sale.	<i>str type d'opération normalisé</i>	parsing
<code>_find_col</code>	Parcourt les colonnes, renvoie la première dont le nom minuscule contient tous les mots-clés.	<i>nom de colonne trouvé, ou None</i>	utilitaire
<code>parse_electronic</code>	Lit les tables HTML, repère celle des transactions, localise chaque colonne, construit les lignes de transactions.	<i>liste de dicts transactions</i>	parsing
<code>run_year</code>	Récupère dépôts d'une année, parse les PTR électroniques, enrichit, écrit CSV, réconcilie Quiver, calcule verdicts et statut.	<i>dict statut de l'année, ou None</i>	orchestration annuelle
<code>run_year.verdict</code>	Classe une ligne de delta : quiver_seul, concordant, quiver_sans_donnee, nous_plus ou quiver_plus_a_verifier.	<i>str verdict de comparaison</i>	validation Quiver
<code>main</code>	Parse les années, charge référentiel et cache Quiver, agrée la session, boucle <code>run_year</code> , agrège et écrit le dashboard.	<i>— (effet de bord : écrit 00_year_status.csv)</i>	orchestration

ocr.py — Orchestrateur OCR papier multi-années : découvre les PTR papier, les OCR-ise via le moteur figé, enrichit et écrit par an.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_image_b64_safe</code>	Cache l'image par hash SHA1 de l'URL complète (anti-collision), retries, redimensionne $\leq$ LONG_EDGE, renvoie PNG base64.	<i>str image PNG encodée en base64</i>	OCR Vision
<code>accept_agreement</code>	GET l'accueil eFD, lit le csrftoken, POST l'agrément de prohibition avec le token.	<i>bool : sessionid ou csrftoken présent dans les cookies</i>	acquisition
<code>fetch_ptr_list</code>	POST paginé l'endpoint data eFD, extrait nom/kind/uuid/date par regex, filtre sur la fenêtre de dates.	<i>DataFrame des dépôts PTR de la fenêtre</i>	acquisition
<code>fetch_paper_html</code>	Renvoie le cache HTML s'il contient les images, sinon GET avec retries, ré-agrèe la session si expirée.	<i>bool : HTML papier disponible avec images</i>	acquisition
<code>discover_index</code>	Agrée, parcourt les années, filtre les PTR papier, collecte uuid/membre/date, écrit l'index CSV.	<i>DataFrame index des rapports papier</i>	acquisition
<code>ocr_one</code>	Fetch le HTML papier puis délègue l'extraction Vision au moteur figé so.extract_report.	<i>(liste transactions, dict cache) du rapport</i>	OCR Vision
<code>build_table</code>	OCR-ise chaque rapport, écarte les lignes-exemple, normalise, résout tickers et identité, applique le schéma.	<i>(df, per_doc, failures, n_example)</i>	OCR Vision
<code>build_table._resolve</code>	Garde le ticker explicite ; sinon tente recover_ticker sur la description ; sinon None.	<i>(ticker, source) tuple</i>	ticker
<code>qa</code>	Affiche un contrôle qualité : compte identité/ticker, dates dans la fenêtre déclarée, owner, montants, asset_type.	<i>— (effet de bord : impression console)</i>	qualité
<code>main</code>	Parse les arguments, vérifie la clé API, construit l'index, exécute le mode index/pilote/full, écrit les CSV.	<i>— (effet de bord : écrit CSV OCR par an)</i>	orchestration

ocr_engine.py — Moteur OCR figé des PTR Sénat scannés : récupère les images, appelle Claude Vision avec cache versionné, normalise.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_api_key</code>	Lit ANTHROPIC_API_KEY dans l'environnement, sinon parse le fichier .env local à la recherche de la clé.	<i>str clé API, ou None</i>	utilitaire
<code>report_gif_urls</code>	Lit le HTML caché du rapport et renvoie toutes les URLs d'images .gif via regex.	<i>liste d'URLs .gif</i>	OCR Vision
<code>_image_b64</code>	Récupère le .gif (cache disque d'abord, sinon GET), redimensionne $\leq$ LONG_EDGE, renvoie PNG base64.	<i>str image PNG encodée en base64</i>	OCR Vision
<code>OcrError</code>	Classe d'exception dédiée signalant un échec d'extraction Vision (corps vide, hérite d'Exception).	<i>— (classe d'exception)</i>	utilitaire
<code>_call_vision</code>	Construit le contenu image+prompt, appelle l'API avec tool forcé, retries sur erreurs, renvoie les transactions du tool_use.	<i>liste de transactions extraites</i>	OCR Vision
<code>extract_report</code>	Sert le cache versionné si valide, sinon calcule la fenêtre, OCR-ise les images par lots, écrit le cache JSON.	<i>(liste transactions, dict cache)</i>	OCR Vision
<code>_infer_asset_type</code>	Déduit l'asset_type par motifs regex : Municipal Security, Corporate Bond, Other, Stock, sinon None.	<i>str type d'actif déduit, ou None</i>	secteur
<code>_fix_year</code>	Si la date lue tombe hors fenêtre PTR, tente de corriger l'année (année de dépôt ou précédente).	<i>str date ISO corrigée, ou entrée brute</i>	qualité

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>normalize</code>	Mappe le code montant en fourchette, nettoie un préfixe propriétaire résiduel, corrige l'année, renvoie la ligne au schéma.	<i>dict transaction normalisée</i>	dédup/schéma
<code>_natural_key</code>	Concatène sept champs métier de la ligne et renvoie leur hash SHA256.	<i>str hash naturel SHA256</i>	dédup/schéma
<code>main</code>	OCR-ise les 4 PTR Blumenthal Q1, écarte exemples, normalise, résout tickers et identité, écrit le CSV golden.	<i>DataFrame OCR (effet de bord : écrit CSV)</i>	orchestration
<code>main._resolve</code>	Garde le ticker explicite ; sinon tente <code>recover_ticker</code> ; sinon <code>None</code> avec source <code>'none'</code> .	<i>(ticker, source) tuple</i>	ticker

fusion.py — Fusion non destructrice digital+OCR par an puis enrichissement global ticker/secteur/date, écrit les tables FINALE.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>date_confidence</code>	Calcule le délai dépôt–transaction ; renvoie plausible (0–75 j), implausible, ou unknown si dates manquantes.	<i>str niveau de confiance de la date</i>	qualité
<code>main</code>	Détecte les années, fusionne digital+OCR sans destruction, enrichit le corpus (ticker/secteur/date), écrit FINAL et dashboard.	— (effet de bord : écrit FINAL.csv + 00_final_status.csv)	fusion

identity.py — Référentiel bioguide, matcher d'identité par chambre, et enrichissement déterministe (ticker, options, dédup) au schéma 23 colonnes du Sénat.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>strip_accents</code>	Décompose en NFKD et retire les caractères combinants pour ôter les accents.	<i>str sans accents</i>	utilitaire normalisation
<code>norm</code>	Ôte accents, passe en minuscules, remplace tout non-lettre par espace, compacte les espaces.	<i>str normalisé en minuscules</i>	utilitaire normalisation
<code>split_name</code>	Prend la partie avant la virgule, écarte les suffixes, renvoie dernier, premier, tokens du milieu.	<i>tuple (last, first, [middles])</i>	identité
<code>load_reference</code>	Charge les YAML législateurs courants/historiques, construit index nom→bioguide et commissions, calcule le flag clé.	<i>tuple de 6 dictionnaires/index</i>	identité référentiel
<code>make_matcher</code>	Fabrique une closure <code>match</code> : override manuel, exact, puis chambre Sénat, titulaire, désambiguïsation prénom.	<i>fonction match(declarant)→ bioguide ou None</i>	identité
<code>recover_ticker</code>	Teste si la description correspond à un motif ticker via regex et renvoie le symbole capturé.	<i>str ticker ou None</i>	ticker
<code>natural_key</code>	Concatène sept champs métier et renvoie leur SHA-256 stable, sans inclure le ticker.	<i>str hash hexadécimal SHA-256</i>	dédup/schéma
<code>enrich</code>	Mappe identité, enrichit ticker, reclasse options, normalise dates, calcule <code>natural_key</code> , <code>occurrence_index</code> , déduplique cross-rapport, réindexe au schéma.	<i>DataFrame au schéma 23 colonnes</i>	identité enrichissement

ticker_resolve.py — Résolution du ticker nom→symbole pour le Sénat en trois étapes : explicite, dictionnaire électronique, puis passe LLM Claude.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>_clean_name</code>	Coupe la description au premier double-espace suivi d'un mot-clé méta via regex, puis trim.	<i>str nom nettoyé</i>	ticker normalisation
<code>_norm_asset</code>	Nettoie, majuscule, retire ticker explicite, ponctuation et suffixes corporatifs, compacte, applique corrections OCR.	<i>str clé d'actif normalisée</i>	ticker normalisation
<code>_explicit_ticker</code>	Cherche un ticker entre parenthèses/tiret en fin de description, vérifie longueur 1 à 5.	<i>str ticker ou None</i>	ticker
<code>_api_key</code>	Charge le .env du dépôt puis le .env courant, renvoie la variable ANTHROPIC_API_KEY.	<i>str clé API (ou vide)</i>	utilitaire LLM
<code>_load_cache</code>	Lit le cache JSON et renvoie ses mappings seulement si prompt_sha et model concordent.	<i>dict mappings ou dictionnaire vide</i>	ticker cache
<code>_save_cache</code>	Crée le dossier parent et écrit model, prompt_sha et mappings en JSON.	— (effet de bord : écrit le cache JSON)	ticker cache
<code>_map_tickers_batch</code>	Appelle l'API Claude avec tool_use forcé, réessaie avec backoff sur erreurs réseau, extrait mappings.	<i>list de mappings nom→ticker</i>	ticker LLM
<code>llm_resolve_tickers</code>	Cible lignes sans ticker, interroge le LLM par lots de 60 (cache versionné), remplit ticker_source=llm.	<i>DataFrame avec tickers LLM remplis</i>	ticker LLM
<code>resolve_tickers</code>	Normalise vacuité, corrige label OCR, applique dictionnaire électronique puis passe LLM, journalise les gains.	<i>DataFrame avec ticker/ticker_source résolus</i>	ticker orchestration

sector_enrich.py — Enrichissement secteur GICS et ETF SPDR par jointure sur le ticker : yfinance primaire, repli LLM, overrides manuels, audit croisé.

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>load_cache</code>	Lit le cache secteur JSON et renvoie ses mappings seulement si pipeline_sha et model concordent.	<i>dict mappings ticker→secteur</i>	secteur cache
<code>save_cache</code>	Écrit model, pipeline_sha et mappings du secteur en JSON sur disque.	— (effet de bord : écrit sector_cache.json)	secteur cache
<code>_yf_sector_one</code>	Interroge yfinance pour un ticker et mappe le secteur Yahoo vers le GICS canonique.	<i>str secteur GICS ou None</i>	secteur yfinance
<code>resolve_yfinance</code>	Résout les tickers via yfinance en plusieurs passes, réessayant les None pour absorber le throttle Yahoo.	<i>dict ticker→secteur GICS</i>	secteur yfinance
<code>_map_sectors_batch</code>	Appelle l'API Claude avec tool_use forcé pour les secteurs, réessaie avec backoff, extrait mappings.	<i>list de mappings ticker→secteur</i>	secteur LLM
<code>resolve_llm</code>	Charge la clé API, interroge Claude par lots de 40, filtre is_listed et secteur valide.	<i>dict ticker→secteur GICS ou vide</i>	secteur LLM
<code>_norm_ticker</code>	Met le ticker en majuscules et supprime les espaces de bordure.	<i>str ticker normalisé</i>	utilitaire normalisation
<code>resolve_sectors</code>	Construit le référentiel : yfinance sur le manquant, LLM (audit complet ou repli), choisit final, sauve le cache.	<i>dict cache complet ticker→secteur</i>	secteur orchestration
<code>enrich_sectors</code>	Résout les tickers du DataFrame, ajoute sector_gics, etf_proxy, sector_source, applique les overrides manuels.	<i>DataFrame avec colonnes secteur ajoutées</i>	secteur application
<code>build_audit</code>	Pour chaque ticker distinct, construit une ligne d'audit avec source, yf, llm et accord croisé.	<i>DataFrame d'audit secteur par ticker</i>	secteur audit
<code>summary</code>	Imprime sources, taux d'accord croisé yfinance↔LLM, extrait de désaccords et distribution sectorielle.	— (effet de bord : impressions console)	secteur audit
<code>main</code>	Charge le CSV final, enrichit secteurs, le réécrit, construit l'audit 06f, imprime la couverture.	— (effet de bord : réécrit CSV + audit)	secteur orchestration CLI

quiver_audit.py — *Validation exhaustive Sénat ↔ Quiver en sept aspects (couverture, ticker, sens, dates, montant, deltas), en lecture seule.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>norm_ticker</code>	Majuscule, écarte valeurs vides/placeholders, retire suffixe option, remplace points et tirets par underscore.	<i>str clé instrument normalisée</i>	validation Quiver normalisation
<code>norm_sense</code>	Réduit l'opération à Purchase, Sale ou Exchange selon l'initiale en minuscules.	<i>str sens réduit</i>	validation Quiver normalisation
<code>_low_bound</code>	Extrait par regex la première borne dollar de la fourchette de montant.	<i>int borne basse ou None</i>	validation Quiver montant
<code>reconcile</code>	Déduplique amendements Quiver, restreint aux sénateurs communs tickérissables, calcule couverture, ticker, accords champ et deltas.	<i>dict de cinq DataFrames d'audit</i>	validation Quiver
<code>_load_inputs</code>	Charge FINAL et cache Quiver, filtre la fenêtre Q1, exclut les délégués hors-chambre.	<i>tuple (df, qwin)</i>	validation Quiver acquisition CLI
<code>main</code>	Charge les entrées, appelle reconcile, écrit les CSV d'audit, imprime couverture, accords et deltas.	— (<i>effet de bord : écrit tables audit + impressions</i>)	validation Quiver orchestration CLI

revalidate_quiver.py — *Re-validation Quiver hors-ligne par année : relit tables et cache, rejoue reconcile avec dédup amendements, met à jour le dashboard annuel.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>verdict</code>	Classe une ligne de delta en concordant, nous_plus, quiver_seul ou quiver_sans_donnee selon ses compteurs.	<i>str verdict de catégorisation</i>	validation Quiver classification
<code>main</code>	Pour chaque année, relit table et Quiver, rejoue reconcile, écrit 07c-f/07, agrège le dashboard de statut.	— (<i>effet de bord : réécrit artefacts + dashboard</i>)	validation Quiver orchestration

census_probe.py — *Phase 0 : recense les PTR Sénat 2020→2026 sur l'eFD, sonde le parser sur HTML ancien et compare l'ordre de grandeur à Quiver.*

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
<code>accept_agreement</code>	GET l'accueil eFD, récupère le csrftoken, POST l'acceptation de l'accord, vérifie la présence des cookies.	<i>bool session ouverte</i>	acquisition eFD
<code>fetch_ptr_list</code>	Pagine la recherche eFD des PTR déposés dans la fenêtre, extrait nom, type, uuid, date, refiltre par date.	<i>DataFrame des PTR listés</i>	acquisition recensement
<code>fetch_report</code>	Renvoie le HTML du rapport depuis le cache disque, sinon le télécharge, le met en cache et le renvoie.	<i>tuple (url, html)</i>	acquisition eFD
<code>amount_midpoint</code>	Extrait les montants dollar et renvoie la moyenne des deux bornes, ou la valeur unique.	<i>float point milieu ou None</i>	parsing montant
<code>norm_op</code>	Normalise le type d'opération en Purchase, Exchange, Sale partielle/complète ou Sale.	<i>str opération normalisée</i>	parsing
<code>_find_col</code>	Renvoie la première colonne dont le libellé minuscule contient tous les mots-clés fournis.	<i>nom de colonne ou None</i>	utilitaire parsing
<code>parse_electronic</code>	Lit les tables HTML, repère la table transactions, localise les colonnes, construit les lignes ou un statut d'échec.	<i>tuple (rows, columns, status)</i>	parsing HTML

Fonction	Ce qu'elle fait (corps réel)	Renvoie	Étape
main	Accepte l'accord, recense par an, sonde le parser sur années anciennes, compare à Quiver, écrit CSV et récap.	— (<i>effet de bord : écrit CSV census/probe + impressions</i>)	orchestration Phase 0

5 Résolution d'identité (priorité #1)

Définition — bioguide_id

Identifiant unique et stable d'un parlementaire (ex. P000197 = Nancy Pelosi). C'est la clé qui relie une transaction à une personne, son parti, son État et ses commissions — et qui évite de **compter deux fois** la même personne écrite de deux façons.

Le problème. Une déclaration donne un nom libre (ñ Hon. Earl L. Carter ž, ñ Buddy Carter ž). Il faut le rattacher au bon `bioguide_id`. **La solution** (`congress_core/identity.py`) : `load_reference` charge le référentiel public *congress-legislators* (current + historical, live avec repli YAML embarqué), construit l'index `name_exact` en y ajoutant des **variantes** (surnom, second prénom, premier mot du nom officiel, dernier mot d'un nom composé) et calcule, par bioguide, l'**année de première entrée en fonction** (`min des terms[] .start`).

La cascade `make_matcher`. `make_matcher` renvoie une fermeture `match(last, first)` qui, après nettoyage des titres (Dr./Hon.) et suffixes (Jr./III/MD), applique dans l'ordre :

1. **Override manuel** (`_MANUAL_BIO`) — un seul cas figé dans le cœur : ñ Nicholas V. Taylor ž = Van Taylor (T000479).
2. **Correspondance exacte** sur (nom, prénom), en testant aussi le **surnom** (table de **31 surnoms** : richard→rick, william→bill...) et le second prénom.
3. **Repli par nom de famille unique** : si un seul bioguide porte ce nom, on le retient.

`enrich_identity` applique ensuite le `matcher` et ajoute `party`, `committee_membership` et `committees_key_flag` (appartenance à une commission clé). Côté Sénat, `senate/identity.py` offre sa propre variante avec désambiguïsation par chambre/titulaire et exclusion des **délégués** non-votants (pour éviter de faux ñ présents chez Quiver, absents chez nous ž).

Résultats (recalculés depuis les FINAL).

Chambre	Lignes	Avec bioguide_id	Bioguides uniques	Noms distincts
House	81 646	81 642 (99,99 %)	256	275
Sénat	8 841	8 841 (100 %)	64	67

Les 4 lignes Chambre sans bioguide sont *une* déclaration manuscrite de 2023 (ñ Ada Norah Henriquez ž, doc 8219483) que le référentiel ne rattache pas — un cas isolé, non un défaut systémique. **256 bioguides pour 275 noms** : l'écart vient de *variantes orthographiques* d'un même élu (ex. ñ Earl L. Carter ž / ñ Buddy Carter ž) toutes rattachées au même identifiant — c'est précisément ce que la résolution d'identité corrige.

6 OCR — lire les scans avec Claude Vision

Les déclarations scannées (image, sans couche texte) sont lues par un **modèle de vision** (`claude-sonnet-4-6`), via le moteur chambre-agnostique `congress_core/vision_ocr.py`.

Deskew (redressement). Beaucoup de scans Chambre sont *couchés*; un modèle lit mal une page tournée. La fonction `detect_rotation` montre la page aux **4 orientations** (`ROT_CANDIDATES=[0,90,180,270]`) et fait *reconnaître* laquelle est droite (tâche de reconnaissance, robuste), puis `rotate_b64_png` redresse géométriquement avant l'extraction (cf. §12 pour l'hypothèse initialement

écartée).

Extraction structurée. `VisionExtractor._call` force `tool_choice` sur l'outil `record_transactions` (le modèle *doit* répondre en JSON structuré), avec `max_tokens` 16 000 et 7 tentatives à *backoff* exponentiel. Garde-fous du prompt : ignorer la **ligne-exemple** pré-imprimée du formulaire, et borner l'année lue à l'année de dépôt.

Cache versionné. `prompt_sha = sha256(prompt + tool + pipeline_tag)[:12]` versionne le cache; `extract_cached` ne rappelle l'API que si le (`prompt_sha`, `model`) a changé, et en cas d'échec partiel **ne re-paie que les lots en erreur** (reprise au batch). Tant que le prompt ne change pas, un re-run coûte **0 appel** — c'est ce qui rend le pipeline incrémental.

Clusters (Chambre, census des 547 PDF scannés).

Cluster	Documents	Traitement
A — tapé droit	74	OCR direct, facile
B — tapé tourné	322	deskew puis OCR (gros volume)
C — manuscrit	151	exclu par défaut (dates manuscrites peu fiables), récupération ciblée

Côté Sénat, pas de clusters : seuls **5 sénateurs** déposent sur papier.

Volumes OCR (recalculés). Chambre : **48 970** lignes; Sénat : **1 680**. L'OCR Chambre est très concentré : **Rohit Khanna** 30 862 lignes (62,9 % de l'OCR), Michael T. McCaul 10 876 (22,2 %), Diana Harshbarger 3 514 (7,2 %) — **top 3 = 92,4 %**, top 10 = 99,0 %. L'OCR Sénat est porté par **Blumenthal** (1 233; 73,4 %), Boozman (379), Burr (52), Feinstein (14), Fetterman (2).

7 Enrichissement : ticker, secteur, montant, ancienneté

Ticker — **trois voies, ajoutées par échecs successifs** (`congress_core/tickers.py`, `llm_resolve.py`) : (1) **explicite** (le symbole est écrit dans la description, `explicit_ticker`); (2) **dictionnaire électronique** (on propage vers le papier les symboles vus dans l'électronique); (3) **passé LLM** (`resolve_names_to_tickers`, filtre `is_equity`, cache versionné) pour le reliquat. Le dictionnaire seul plafonne (~46 %); le LLM porte la couverture à **~90 %** sur l'OCR. Les actifs non cotés (obligations, munis) restent `ticker=None` à **dessein**, pas par échec.

Secteur GICS→ETF SPDR (`congress_core/sector_enrich.py`, `enrich_sectors`) : résolution hybride `yfinance` (factuel) → repli **LLM** (enum GICS strict) → overrides manuels, avec cache d'audit traçant qui a tranché (`sector_source`). Mapping 1 :1 des **11 secteurs** vers les *Select Sector SPDR* :

Energy XLE · Materials XLB · Industrials XLI · Cons. Disc. XLY · Cons. Staples XLP · Health Care XLV ·
Financials XLF · IT XLK · Comm. Services XLC · Utilities XLU · Real Estate XLRE.

Montant — *midpoint* (`amounts.py`, `amount_midpoint`) : les déclarations donnent des **fourchettes** (ex. \$15 001–\$50 000); on retient le **milieu** (32 500). Le type d'opération est normalisé par `operation_type_from_code` (P→Purchase, S→Sale (Full/Partial), E→Exchange).

Ancienneté (`enrich_tenure.py`) : `years_in_office = année(transaction) – année(première entrée en fonction)`, calculée offline depuis les **terms** embarqués et **appendue octet-à-octet** en dernière étape (idempotent). Renseignée à **100 %** sur les deux chambres.

8 La table finale — schéma et contrat à 12/12

La table FINAL compte **28 colonnes** (27 d'assemblage + `years_in_office` appendue). Les **12 champs à métier** exigés par Ramify sont *exactement* la constante `schema.CORE_FIELDS` :

`declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date`
`· ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type`

Nuance d'honnêteté : $\hat{n}$ 12 champs *présents* $\hat{z} \neq \hat{n}$ 12 champs *sans valeurs manquantes* $\hat{z}$

Les 12 champs sont **présents** dans le schéma des deux chambres. Mais trois d'entre eux ont des **trous légitimes**, par nature et non par défaut d'extraction : `ticker/sector_gics/etf_proxy` sont *vides* pour les actifs non cotés (obligations, *munis*, fonds privés) — il n'existe pas de symbole boursier à leur attribuer ; et `committee_membership` est un **snapshot courant** (pas l'historique daté). Les champs structurellement toujours renseignables (`declarant_name`, `chamber`, `party`, `transaction_date`, `disclosure_date`, `operation_type`, `amount_midpoint`) sont quasi-complets. Nous assumons cette distinction plutôt que de la masquer.

Taux de remplissage global (rows-based, recalculés depuis les FINAL).

Champ	Chambre	Sénat
<code>ticker</code>	85,3 %	71,4 %
<code>sector_gics / etf_proxy</code>	83,2 %	62,1 %
<code>asset_type</code>	91,1 %	97,3 %
<code>committee_membership</code>	86,6 %	65,6 %
<code>years_in_office</code>	100 %	100 %
<code>bioguide_id</code> (identité)	99,99 %	100 %

Le schéma complet des 28 colonnes (sens + origine de chacune) est en Annexe 16.2.

9 Rapport de qualité & métriques

Le module `congress_core/quality.py` (`build_report` $\rightarrow$ `_figures`) calcule **5 contrôles** et génère les 4 figures ci-dessous — **en lecture seule des FINAL, sans aucun appel API** (matplotlib Agg). Tous les chiffres ci-dessous en sont issus.

9.1 Les cinq contrôles

(a) **Cohérence des dates** (`date_coherence`, `disclosure` $\geq$ `transaction`) : **99,8 %** cohérentes côté Chambre, **100,0 %** au Sénat (dates parseables 99,8 / 99,9 %). Les rares incohérences sont des divulgations amendées/antidatées réelles, non des bugs.

(b) **Délai légal** (`delay_buckets`) : **86,9 %** des dépôts Chambre et **84,9 %** au Sénat tombent dans les **45 jours** légaux (délai médian **28 jours**) ; >75 j : 7,7 / 12,3 %.

(c) **Distribution des montants** (`amount_distribution`) : médiane **8 000 \$**, quartile haut **32 500 \$**, 90^e centile 75 000 \$ (*midpoint* des fourchettes).

(d) **Couverture par membre** (`coverage_per_member`) : **320** déposants distincts, **206** avec ≥ 10 transactions (éligibles au backtest), dont **150** actifs sur ≥ 3 années.

(e) **Achats sans sortie déclarée** (`unmatched_purchase_rate`) : **22,5 %** (Chambre) / **39,6 %** (Sénat) des achats tickés n'ont pas de vente ultérieure déclarée — ces positions seraient fermées de force à +12 mois dans la stratégie.

9.2 Les quatre figures

9.3 La couverture par an — et la baisse Sénat 2025–26

Le **global** masque une dynamique : la couverture ticker/secteur du Sénat **chute en 2025–26**. La présenter **par an** est plus honnête.

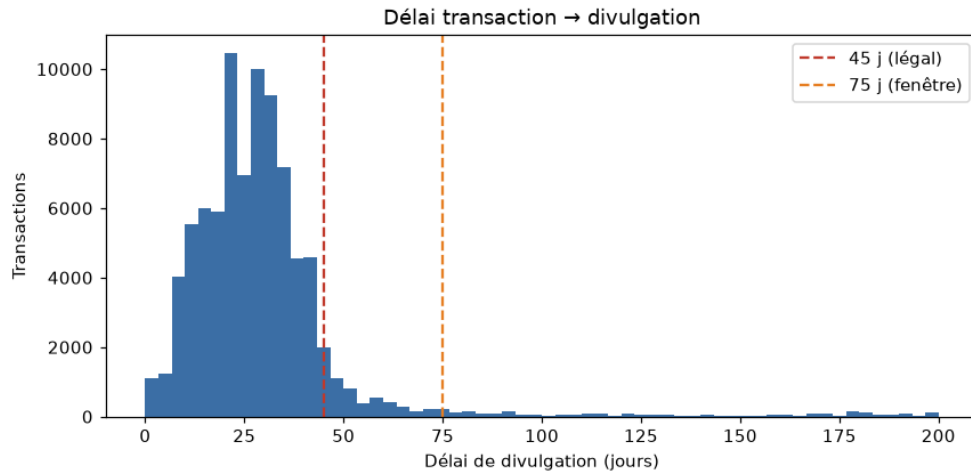


FIGURE 1 – Délai disclosure – transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `congress_core/quality.py` (`_figures`, source `delay_buckets`) — lecture seule des FINAL, aucun appel API.

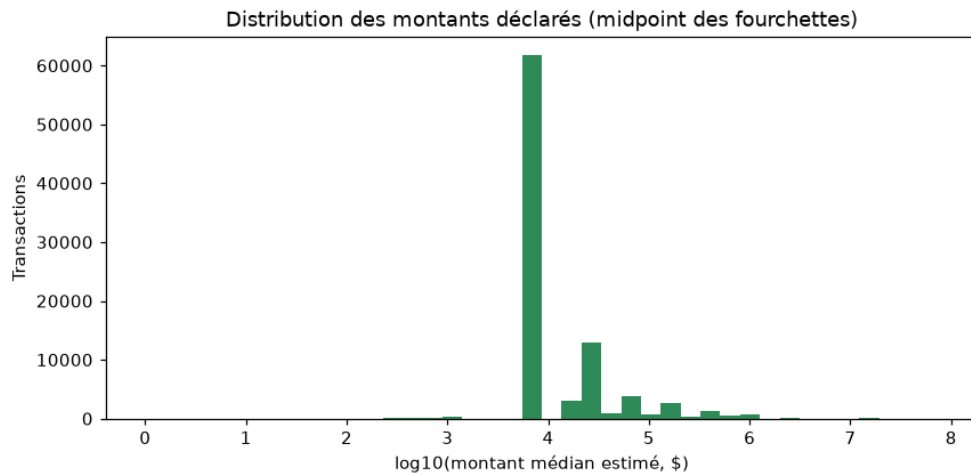


FIGURE 2 – Distribution de `amount_midpoint` en échelle `log10` (midpoint des fourchettes). Générée par `quality.py` (`_figures`, source `amount_distribution`).

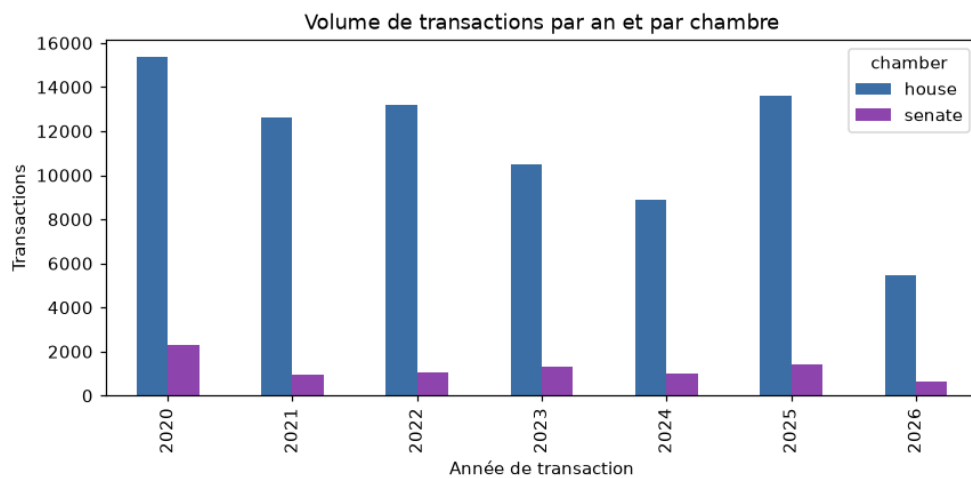


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `quality.py` (`_figures`). On lit la montée du volume OCR et la part Chambre dominante.

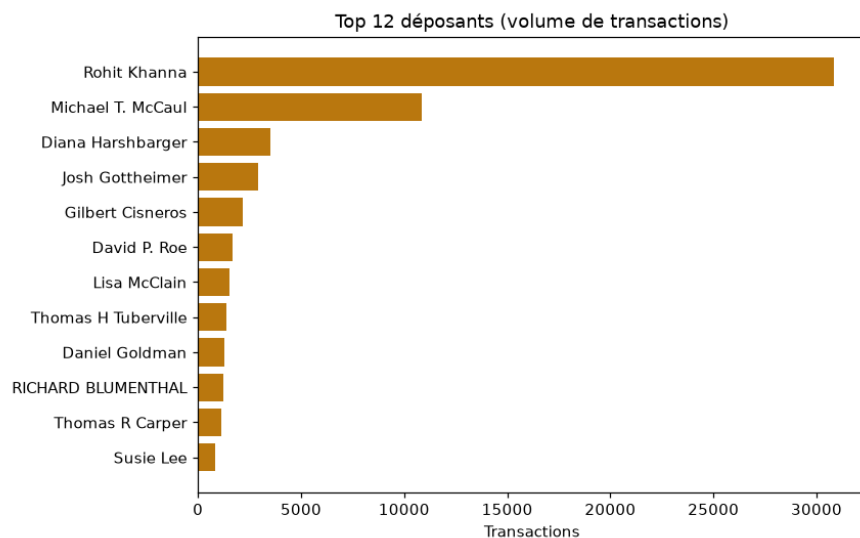


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `quality.py` (`_figures`, source `coverage_per_member`). La domination de quelques déposants-papier (Khanna, McCaul) y est visible.

Année	Chambre		Sénat	
	ticker %	sector %	ticker %	sector %
2020	86,5	83,4	88,1	80,5
2021	87,3	85,8	80,9	70,1
2022	87,1	85,2	79,7	65,3
2023	78,9	76,6	77,7	69,5
2024	81,3	78,3	68,0	59,2
2025	87,8	86,3	45,4	37,2
2026	85,4	84,4	43,7	35,7

Explication. En 2025–26, la part de **papier OCR** et surtout d'**obligations municipales / Treasuries** (non cotées, donc `ticker=None` légitimement, ex. Blumenthal) augmente fortement au Sénat. Le global (71,4 / 62,1 %) est *tiré vers le haut* par les années antérieures. Ce n'est pas une dégradation d'extraction mais un **changement de composition des actifs**.

9.4 Fiabilité des dates OCR (`date_confidence`)

`date_confidence` = drapeau *plausible/implausible* selon la fenêtre 75 j entre transaction et dépôt ; il n'est calculé que pour l'**OCR** (NaN sur le digital). Côté Chambre, **95,3 %** des lignes OCR sont *plausibles* (2 291 implausibles) ; le creux **2021 à 79,6 %** est *entièrement* Diana Harshbarger (1 389 lignes sur 2 documents manuscrits denses) — anomalie **localisée**, pas systémique. Côté Sénat : 98,5 % plausibles (18 implausibles, le lot Perdue 2021 = divulgations tardives *réelles*).

10 Validation externe (Quiver)

Méthode — deux axes (Quiver jamais réinjecté)

- (1) **Comptes par déposant** (per-lot) : nombre de lignes chez nous vs chez Quiver, par personne.
- (2) **Recouvrement transaction-niveau** : part des transactions Quiver retrouvées, via la clé `bioguide | ticker | date | type`. La date appariée est le *Traded* de Quiver (le *Filed* n'apparie que ~1 %). **Couverture** = transactions Quiver retrouvées / total Quiver.

Chambre — clé $\acute{n}$ brute $\acute{z}$ (sans tolérance de date), `validate_quiver_house`. Recalcul depuis le cache transaction-niveau :

Année	Couverture %	Quiver retrouvées	<code>only_quiver</code>	vrais-absents
2020	85,0	7 106	1 253	16
2021	88,6	5 221	672	14
2022	80,7	6 061	1 452	29
2023	81,0	6 128	1 442	65
2024	85,8	4 670	772	40
2025	91,3	9 835	938	58
2026	88,0	4 115	559	27
Global	85,9	43 136	7 088	249 → 9

La couverture globale **85,9 %** (43 136 / 50 224) est un **plancher** : la clé sur date *brute* sous-estime sur les dates OCR bruitées. L'accord de **sens** (achat/vente) et de **montant** sur les transactions appariées est **~93 %**. Les **249 $\acute{n}$ manquants bruts $\acute{z}$** se réduisent à **9 vrais-absents** après explication des artefacts (tolérance de date ± 1 j, amendements, options/obligations) : 2021 Malinowski $\times 1$; 2023 Schrader $\times 3$ (papier, cluster C exclu) ; 2024 $\acute{n}$ Calhoun $\acute{z}\times 2$ (étiquetage Quiver de J. James) ; 2025 Gottheimer $\times 1$ et 2026 Pelosi $\times 2$ (déposés *après* notre snapshot). **Aucun bug d'extraction.**

Sénat — clé normalisée, `quiver_audit.reconcile` (07c–07f). Couverture digitale **98–100 %/an** (99,4 / 98,0 / 99,4 / 99,7 / 99,6 / 99,8 / 100,0 %), accord de sens **~95,7 %**, date (*Traded*) **~99,4 %**, montant **~93,2 %**. **554 amendements** Quiver dédupliqués (un trade re-déposé y apparaît plusieurs fois). **0 vrai raté** après audit adversarial relisant les PTR bruts.

Le constat $\acute{n}$ OCR = source unique $\acute{z}$

Quiver — comme Kadoa et *House Stock Watcher* — **ne voit aucun scan** : 0 ligne pour les gros déposants-papier (Khanna, Harshbarger, Blumenthal). Le terme `only_ours` de la Chambre (**~16 000** transactions) est donc majoritairement le **papier que les agrégateurs ne voient pas** — pas une erreur, mais notre **valeur ajoutée**, validée en interne (cohérence, stabilité run-à-run, `date_confidence`). Le module `crosscheck.py` matérialise ce statut par déposant (`quiver_validable / ocr_unique / digital`).

Lever l'ambiguïté du $\acute{n}$ 35–74 % $\acute{z}$. On lit parfois cette fourchette : c'est la couverture Quiver du **numérique *seul***, par année — large parce que la part de déposants-papier varie d'une année à l'autre. Ce n'est *pas* un verdict sur le FINAL : avec l'OCR, le FINAL monte à **81–91 %** (85,9 % global).

11 Assurance qualité & reproductibilité

Pourquoi pas $\acute{n}$ rejouable hors-ligne $\acute{z}$. Le scraping (Clerk/eFD) exige le réseau et l'OCR exige l'API Vision ; seuls les **artefacts figés** (tables CSV, index XML, caches, référentiels YAML) sont embarqués. Rejouer une année digitale produirait une table *vide* (les PDF lisibles ont été parsés puis écartés). On prouve donc la correction par **reproduction depuis les colonnes figées**, preuve plus forte car déterministe et isolée des aléas réseau/OCR.

- **Golden (photo octet-à-octet).** Empreintes SHA-256 de toutes les sorties : **108** fichiers Chambre, **69** Sénat. `check_golden.py / senate_check_golden.py` doivent afficher $\acute{n}$ ZÉRO ÉCART $\acute{z}$.
- **Reproductions fonction-par-fonction.** `test_senate_repro` reconstruit, depuis les seules colonnes figées, `natural_key_hash` **8 841/8 841**, l'identité **67/67**, le ticker **65/65** ; côté Chambre, `test_schema/test_amounts_tickers/test_identity/ test_tenure` reproduisent clé, montant, type d'actif, bioguide et ancienneté.
- `test_vision_sha / test_incremental` : le `prompt_sha` est stable (déplacer l'OCR n'invalide pas le cache) et un 2^e run OCR fait **0 appel** Vision.

- `audit_metrics.py` recompute toutes les métriques des deux chambres et *asserte* les invariants — c'est la source de vérité chiffrée de ce rapport.

12 Hypothèses explorées puis révisées

La valeur *à* recherche *à* tient autant aux choix *révisés* qu'aux résultats. Pour chacun : **hypothèse de départ** → **problème constaté** → **décision finale**.

Deskew OCR. *Départ* : demander au modèle *à* de combien faut-il tourner ? *à*. *Problème* : tâche spatiale peu fiable — il répond presque toujours *à* 90 *à*. *Décision* : lui montrer la page aux **4 rotations** et lui faire *reconnaître* la droite (tâche de reconnaissance). La concordance OCR passe de **59,7 %** à **≈69 %**.

Déduplication. *Départ* : dédupliquer sur la seule clé naturelle. *Problème* : cela écrase des **lots multi-comptes réels** (ex. Khanna déclarant 3 fois le même titre via 3 comptes). *Décision* : `occurrence_index` (rang du lot intra-dépôt) → dédup **non destructrice** sur le *couple* (`natural_key_hash`, `occurrence_index`).

OCR par échantillon d'abord. Avant le run complet des 547 PDF, `house/echantillon.py` OCR-ise un **échantillon stratifié** des clusters A/B/C pour *mesurer* la qualité et durcir le prompt — au lieu de découvrir les modes d'échec en production.

Cluster C manuscrit. *Départ* : tout OCR-iser. *Problème* : les **dates manuscrites** sont trop incertaines pour une stratégie datée. *Décision* : **exclure C** par défaut (récupération ciblée). Une sonde **Opus vs Sonnet** sur C a confirmé qu'**Opus ne corrige pas le goulot** (les dates), pour un coût $\approx \times 3$ — on garde Sonnet, C reste exclu.

Fenêtre de plausibilité de date. Le STOCK Act vise ≈ 45 j, mais on tolère **75 j** (`date_confidence`) pour absorber le bruit OCR — et l'on note explicitement que cette colonne n'est calculée que sur l'OCR (les contrôles (a)/(b) sont, eux, calculés directement sur les dates).

Ticker en trois voies. Ajoutées *par échecs successifs* : explicite, puis dictionnaire électronique, puis LLM (filtre `is_equity` pour ne *jamaïs* tickeriser une obligation/muni).

Validation Quiver, deux régimes de clé. **Chambre** = clé *à* brute *à* (reproduit l'historique, couverture = plancher) ; **Sénat** = clé *à* normalisée *à* (**reconcile**) avec dédup des amendements. On *n'unifie pas* les clés : chaque chambre garde sa sémantique exacte.

13 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** (dates trop incertaines) ; récupération seulement ciblée — choix assumé, pas une lacune cachée.
- **Validation papier externe impossible** : aucun agrégateur ne voit les scans → pas de taux d'exactitude *externe* sur le papier (seulement des contrôles internes).
- **Appariement Quiver Chambre sans tolérance de date** (clé brute) : la couverture **85,9 %** est un **plancher**, pas une borne haute.
- **ticker/sector_gics/etf_proxy structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut.
- **Commissions non *à* point-in-time *à*** : `committee_membership` est un *snapshot* courant, pas l'historique daté — un croisement *à* conflit d'intérêt à la date du trade *à* serait approximatif.
- **date_confidence = heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** (NaN sur le digital) ; ce n'est pas une vérité-terrain.

14 Périmètre & suite

La **couche données (Semaines 1–2)** décrite ici est **livrée** : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest (Semaines 3–4)** — copy-trading actions puis ETF sectoriels — constituent un **travail de recherche séparé, en cours**, hors-périmètre de ce rapport.

15 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est **identifiée** (99,99 / 100 %), **enrichie** (table 12/12 sur les deux chambres) et **validée** contre Quiver sans jamais le réinjecter (85,9 % Chambre, 98–100 % Sénat ; 9 et 0 vrais-absents). Le **papier scanné**, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne. La correction est **figée** (golden octet-à-octet) et **reproductible** (reproductions fonction-par-fonction). Surtout, le rapport assume ses **nuances** — 12 champs présents ȳ n’est pas 12 champs sans manquants ȳ, la couverture Sénat 2025–26 baisse pour des raisons de composition d’actifs, et la couverture Quiver Chambre est un plancher. C’est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

16 Annexes

16.1 Annexe A — les 44 fichiers .py en un coup d’œil

Fichier	Rôle
congress_core/ (15, dont __init__)	
schema.py	clé naturelle (7 champs) + dedup non destructrice
identity.py	nom → bioguide + parti/commissions/ancienneté
amounts.py	fourchette → milieu, code → opération
tickers.py	symbole boursier + type d’actif
vision_ocr.py	OCR Claude Vision + cache versionné + deskew
llm_resolve.py	cache LLM versionné (nom→ticker)
quiver.py	validation externe (Chambre brute / Sénat normalisée)
crosscheck.py	statut par déposant (OCR-unique vs Quiver)
sector_enrich.py	ticker → GICS → ETF (yfinance + LLM)
reporting.py	conventions de sortie, QA, Excel
paths.py	chemins (DataRoot) + secrets
pipeline.py	l’orchestrateur (6 étapes + qualité)
quality.py	rapport de qualité (5 contrôles + figures)
enrich_tenure.py	years_in_office (append octet-préservant)
house/ (4, dont __init__)	
digital.py	PTR électroniques → 06..._transactions
ocr.py	scans → Vision → fusion → 06..._FINAL
echantillon.py	mesure stratifiée OCR (pilote/audit)
senate/ (11, dont __init__)	
digital.py	eFD électronique → 06..._transactions
ocr.py	papier eFD → Vision → 06b
ocr_engine.py	moteur OCR figé (Q1) réutilisé
fusion.py	fusion digital+OCR + enrichissement → FINAL
identity.py	référentiel + matcher Sénat
ticker_resolve.py	ticker en 3 étapes (explicit/dict/LLM)
sector_enrich.py	secteur GICS→ETF (Sénat)

Fichier	Rôle
quiver_audit.py	réconciliation Quiver (07c-07f)
revalidate_ quiver.py	re-validation Quiver offline
census_probe.py	Phase 0 : census + sondage parser

tests/regression/ (14)

golden (build_golden/check_golden + Sénat) · reproductions (test_schema, test_identity, test_ -
amounts_tickers, test_senate_repro, test_vision_sha, test_incremental, test_tenure, test_ -
crosscheck) · audit_metrics · _apply_house_sector

16.2 Annexe B — le schéma FINAL (28 colonnes)

#	Colonne	Sens	Remplie par
1	bioguide_id	identifiant officiel du membre	identity
2	declarant_name	nom tel que déposé	parse / OCR
3	chamber	house / senate	enrich_identity
4	party	parti	Reference.party
5	state_district	état + district	03_ptr_index
6	committee_ membership	commissions (liste)	Reference.committees
7	committees_key_flag	commission n° clé ?	is_key_committee
8	transaction_date	date de la transaction	parse / OCR
9	disclosure_date	date de divulgation (dépôt)	03_ptr_index
10	ticker	symbole boursier	tickers / llm_ resolve
11	asset_description	libellé de l'actif	parse / OCR
12	asset_type	type d'actif	infer_asset_type
13	sector_gics	secteur GICS	sector_enrich
14	etf_proxy	ETF SPDR du secteur	GICS_TO ETF
15	operation_type	Purchase / Sale / ...	operation_type_ from_code
16	amount_range	fourchette déclarée	parse / OCR
17	amount_midpoint	milieu de la fourchette	amount_midpoint
18	amount_split_flag	lot multi-actifs ?	finalize
19	owner	propriétaire (self/spouse...)	normalize
20	doc_id	identifiant du PTR	03_ptr_index
21	source_url	URL de la source	03_ptr_index
22	natural_key_hash	clé de dédup (7 champs)	schema
23	occurrence_index	rang du lot intra-dépôt	add_occurrence_index
24	provenance	*-electronic / *-ocr	pipeline
25	date_confidence	plausible / implausible (75 j)	date_confidence
26	ticker_source	explicit / elec_dict / llm / none	tickers / llm
27	sector_source	yfinance / llm / manual	sector_enrich
28	years_in_office	ancienneté à la date du trade	enrich_tenure

*L'ordre exact des colonnes diffère légèrement entre Chambre et Sénat (date_confidence plus tôt au Sénat); le **contenu garanti** — les 12 champs n° métier — est identique.*

16.3 Annexe C — valeurs vérifiées (recalculées depuis les FINAL)

Indicateur	Chambre	Sénat
Transactions FINAL	81 646	8 841
dont digital / OCR	32 676 / 48 970	7 161 / 1 680
Identité (bioguide_id)	99,99 % (256/275)	100 % (64/67)
ticker	85,3 %	71,4 %
sector_gics / etf_proxy	83,2 %	62,1 %
asset_type	91,1 %	97,3 %
committee_membership	86,6 %	65,6 %
years_in_office	100 %	100 %
Quiver (transaction-niveau)	85,9 %/an (9 vrais-absents)	98–100 %/an (0 vrai raté)
Golden (zéro écart)	108 fichiers	69 fichiers
Qualité (depuis quality.py)		
Cohérence des dates	99,8 %	100,0 %
Dépôts ≤ 45 j (médiane 28 j)	86,9 %	84,9 %
Montants — médiane / p75	8 000 \$ / 32 500 \$	
Déposants (≥ 10 trades / ≥ 3 ans)	320 (206 / 150)	
Achats sans sortie	22,5 %	39,6 %

Total : 90 487 transactions, 7 ans (2020–2026). Concentration : Rohit Khanna $\approx 30\,862$ transactions (top 3 Chambre $\approx 92,4\%$ de l'OCR).

16.4 Annexe D — reproduire / lancer

```
# Installation
python3.12 -m venv .venv && ./venv/bin/pip install -e .

# Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence
./venv/bin/python -m congress_core.pipeline --years 2020-2026
./venv/bin/python -m congress_core.pipeline --years 2024 --dry-run

# Rapport de qualite (offline, lecture seule des FINAL)
# -> docs/RAPPORT_QUALITE.md + docs/quality/*.png
./venv/bin/python -m congress_core.quality

# Filet "zero changement" (doit afficher ZERO ECART)
./venv/bin/python tests/regression/check_golden.py      # Chambre, 108
./venv/bin/python tests/regression/senate_check_golden.py # Senat,    69
./venv/bin/python tests/regression/test_senate_repro.py  # reproductions
```

Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.